MyGameList

1. Application Overview

Project Name

- MyGameList

Application Goal

MyGameList is a web application that allows users to create and manage a personal list of video games they have played, are currently playing, or plan to play.

Users can rate games, write notes or reviews, upload cover images or screenshots, organize games by status, and browse public game lists from other users.

The primary value of the application is to help players track their gaming history and share recommendations with others.

Target Audience

The application is intended for:

- Gamers who want to track completed games
- Players managing a gaming backlog
- Users who want to review or rate games
- Friends or communities sharing game recommendations
- Administrators maintaining the platform

2. User Roles and Access

User Role	Description	Key Permissions
Guest	An unregistered public visitor.	View public pages, browse public user lists, search games, register an account, log in.
Gamer	A registered user with a personal game list.	Add games to their list, edit/delete their own entries, rate games, write reviews, upload profile picture, manage profile, view other public profiles.
Moderator	A trusted user who helps manage public content.	Review reported content, hide inappropriate reviews, manage public comments, view moderation dashboard.
Administrator	Full system manager.	Manage all users, manage all game entries, delete inappropriate content, access admin dashboard, view system health information.

3. Core Use Cases and Features

A. Account Management and Access

User Registration

Required Information

- Username
- Email address
- Password
- Confirm password
- Optional display name
- Agreement to terms of service

Registration Process

1. User opens the registration page.
2. User enters username, email, and password.

3. Application validates the input.
4. Password is securely hashed before storage.
5. A new user account is created.
6. Optional email verification is sent.
7. User is redirected to the login page or automatically logged in.

User Login

Registered users can access their account through a login form.

Required Inputs

- Email or username
- Password

Login Process

1. User enters login credentials.
2. Backend checks if the account exists.
3. Password is compared against the stored password hash.
4. If valid, a secure session or authentication token is created.
5. User is redirected to their dashboard.
6. If invalid, an error message is displayed.

User Profile Update

Users can update their profile from the account settings page.

Editable Fields

- Display name
- Bio
- Platforms (pc, ps, xbox, nintendo ect.)

- Favorite genre
- Profile visibility
- Email address
- Password
- Profile picture

Profile Update Process

1. User opens account settings.
2. User edits profile fields.
3. User submits the form.
4. Backend validates changes.
5. Database is updated.
6. User receives a success notification.

Password Change Process

Required inputs:

- Current password
- New password
- Confirm new password

The user must enter their current password before changing to a new one.

B. Content and File Interaction

In this application, games are selected from a **public, managed game catalog**. Users do not create new global game records directly. Instead, they browse or search the catalog and add existing games to their personal list.

The catalog is managed by administrators or imported from an approved external game database API.

Public Game Catalog

The application contains a shared catalog of video games.

Each game in the catalog represents one official game record that can be added to many users’ personal lists.

Catalog Purpose

The public catalog allows the system to:

- Avoid duplicate game records
- Keep game information consistent
- Display shared metadata such as title, cover image, release date, and platforms
- Allow users to search and discover games
- Connect user ratings and reviews to the same game record

Game Catalog Data

Each catalog game may include:

- Game ID
- Title
- Description
- Cover image
- Developer
- Publisher
- Release date
- Genres
- Available Platforms
- Age rating
- Official website

- submitted at: timestamp
- last updated: timestamp

Example Catalog Entry

```
{
    "Title": "Hollow Knight",
    "Developer": "Team Cherry",
    "Publisher": "Team Cherry",
    "Release Date": "2017-02-24"
    "Genres": ["Metroidvania", "Action", "Platformer"]
    "Platforms": ["PC", "Nintendo Switch", "PlayStation", "Xb
ox"]
}
```

Who Can Manage the Catalog?

Guests

Guests can:

- Browse public catalog pages
- Search for games
- View public game details

Gamers

Standard users can:

- Browse the public game catalog
- Add catalog games to their personal list
- Mark games as completed, playing, dropped, or planned
- suggest games to the catalog (with a trusted source)

Moderators

Moderators can:

- Review suggested catalog changes
- Flag incorrect game metadata

Administrators

Administrators can:

- Add new games to the public catalog
- Edit catalog game information
- Delete or archive catalog entries
- Import metadata from approved external APIs
- Review user-submitted game suggestions
- Manage cover images and metadata

Media/File Submission

Users can upload images related to their profile or game entries.

Supported Upload Types

- Profile picture
- Custom game cover image
- Screenshots

Allowed File Types

- `.jpg`
- `.jpeg`
- `.png`
- `.webp`

File Size Limit

Example:

- Profile picture: maximum 2 MB
- Game screenshots: maximum 5 MB each

Required Inputs

- File
- File name
- Upload type
- Related catalog Id (for screenshots and game covers)

File Storage

Files may be stored in:

- Cloud object storage in production
- Local development storage during testing

Upload Process

1. User selects an image file.
2. Frontend checks file type and size.
3. Backend validates the file again.
4. File is renamed using a safe generated name.
5. File is stored in the upload storage location.
6. Database stores the file URL or storage key.
7. Image becomes available on the relevant page.

Content Display and Management

How User-Submitted Content Is Displayed

Game entries are displayed in several places:

1. **User Dashboard**

- Shows the user’s own list.
- Allows filtering by status, rating, platform, or genre.

2. **Public Profile**
    - Shows public game entries.
    - May hide private entries depending on user settings.

Example Game List Display

Each game card may include:

- Cover image
- Catalog Id
- Played Platform
- Status
- Hours played
- Completion date

Editing Own Content

Users can edit their own game entries.

Editable Fields

- Status
- Platform
- Hours played
- Completion date
- Tags
- Visibility

Edit Process

1. User opens their game entry.

2. User clicks **Edit**.
3. User changes the desired fields.
4. User submits the update.
5. Backend verifies ownership.
6. Updated data is saved.

Deleting Own Content

Users can delete their own game entries.

Delete Process

1. User clicks **Delete** on a game entry.
2. Application shows a confirmation dialog.
3. User confirms deletion.
4. Backend verifies ownership.
5. Entry is deleted or soft-deleted.
6. User receives a confirmation message.

Instead of permanently removing the row, mark it as:

```
deleted = true
```

External Data Retrieval

The application can fetch video game metadata from an external game database API.

Example external services:

- RAWG Video Games Database API
- IGDB API
- Steam DB API

- Giant Bomb API

Feature: suggest games to the catalog

When a user searches for a game title, the backend requests metadata from an approved external API.

User Input

- Game title
- Platform, optional
- Release year, optional

Backend Process

1. User searches for a game.
2. Backend sends request to an approved external game metadata API.
3. API returns possible matches.
4. User selects the correct game.
5. Application open a request to imports selected metadata.

Imported Metadata

- Game title
- Cover image URL
- Developer
- Publisher
- Release date
- Genre
- Platforms
- Description

C. System Resource Access

Administrative System Log Viewing

Administrators may need to view application logs from the admin dashboard.

Feature: View Application Logs

Administrators can view recent logs related to:

- User registration
- Login attempts
- Failed uploads
- Content reports
- Server errors
- API errors

Admin Inputs

Instead of allowing admins to enter raw server file paths, the system should provide safe log categories.

Example log categories:

- Authentication logs
- Upload logs
- Error logs
- Moderation logs
- API request logs

Safe Process

1. Administrator opens the admin dashboard.
2. Administrator selects a log category.
3. Backend maps the category to a predefined safe log file.

4. Backend reads only from approved log locations.
5. Logs are displayed in the admin interface.

Example Safe Log Mapping

```
auth_logs -> /var/app/logs/auth.log
error_logs -> /var/app/logs/error.log
upload_logs -> /var/app/logs/uploads.log
```

Security Requirements

- Do not allow arbitrary file paths from users.
- Do not expose sensitive secrets.
- Mask passwords, tokens, API keys, and session IDs.
- Restrict access to administrators only.
- Keep an audit trail of which admin viewed which logs.

Batch/Sequential Processing

Feature: Bulk Update Game Status

Users may update multiple games in their list at once.

Example use cases:

- Mark several games as completed.
- Move multiple games from “Want to Play” to “Currently Playing.”
- Add a tag to selected games.
- Change visibility for several entries.

User Inputs

- Selected game entry IDs
- New status

- Optional tag
- Optional visibility setting

Process

1. User selects multiple games from their list.
2. User chooses a bulk action.
3. Application shows a confirmation screen.
4. User confirms.
5. Backend verifies ownership of every selected game.
6. Updates are processed in a controlled transaction.
7. User receives a summary of the results.

Example

A user selects:

- Hollow Knight
- Celeste
- Hades

Then applies:

```
Status: Completed
Tag: Favorites
Visibility: Public
```

Why Sequence Matters

The application should ensure that:

- Only the user’s own games are updated.
- All updates are completed successfully or rolled back.
- The final game counts are accurate.
- Notifications and activity feed items are created only after successful updates.

Recommended Technical Approach

Use database transactions.

Example:

```
BEGIN TRANSACTION

Update selected game entries
Update user statistics
Create activity feed records

COMMIT
```

If any step fails:

```
ROLLBACK
```

D. Data Visibility and Flow

System Messages

Users receive in-app notifications for important actions and updates.

Types of Messages

- Game added to list
- Game entry updated
- Game deleted
- Profile updated
- Password changed
- Upload failed
- Moderator removed content
- Admin announcement

Notification Data Fields

Each message may include:

- Notification Id
- User Id
- Message title
- Message body
- Action Type, optional
- Action metadata, optional
- Read/unread status
- Created timestamp

Example Notification

```
{
  "notificationId": "unique-id-123",
  "userId": "user-uuid-456",
  "messageTitle": "Game Added",
  "messageBody": "Elden Ring was added to your Completed lis
t.",
  "actionType": "game_entry_updated",
  "actionMetadata": {
    "CatalogId": "a74f37ba-ab2f-46c0-93d1-a0ac216f7df7",
    "status": "Completed"
  },
  "shown": true,
  "createdTimestamp": "2026-05-05T17:00:00+03:00"
}
```

Notification Display

Notifications can appear in:

- Bell icon menu

- User dashboard
- Webhook, myabe

Request Integration

Feature: Delete Game Entry

A user can delete a game entry from their list.

Required Request Data

- Game entry ID
- Authenticated user session (JWT)

Example Request Flow

1. User clicks **Delete** on a game entry.
2. Confirmation modal appears.
3. User confirms.
4. Frontend sends a POST or DELETE request.
5. Backend verifies:
    - User is logged in.
    - User owns the game entry.
6. Game entry is soft-deleted.
7. User receives success message.

Example Endpoint

```
DELETE /api/game-entries/:id
```

Example Response

```
{
  "success": true,
  "message": "Game entry deleted successfully."
}
```

4. Technical Stack

Frontend Technologies: React, HTML, CSS, and JavaScript.

Backend Technologies/Language: FastAPI with Python.

Database: MySQL

Development Environment/Hosting: docker compose for development and cloud hosting